

RockFall - developer guide

RockFall is the smallest complete game on the engine, so it is the best one to read first. Read `docs/GUIDE.md` (the engine and stack guide) for the technology; this file is about this game.

The game: you steer a pebble along the bottom by dragging a finger (or moving the mouse). Rocks fall; touch one and the run ends. The score is the seconds you survived. Rocks come faster and more often over time. After you die you may "Watch ad to continue" **once per run**: the field clears and you get 1.5 s of invulnerability.

- Branch `rockfall`, folder `RockFall`, app ID `com.maxmartin.rockfall`
- Saved data: your best score, in `localStorage` key `rockfall.best`
- Dev URL flag: `?seed=N` - every run uses seed N (same rocks every time)

Files

```
src/game/
  index.ts          the FLOW: phase machine title/playing/paused/over/ad; best score; revive-with-ad
  info.ts           GAME_ID, title text, the little rock logo
  sim/              PURE rules (no DOM, no Pixi, no clock)
    config.ts        ALL tuning numbers
    difficulty.ts     fallSpeed(t) and spawnInterval(t): how it gets harder
    game.ts          the player, rocks, spawning, movement, collisions  <- the game rules
    step.ts          the lifecycle: createState, step, revive, scoreOf, events
    types.ts         GameState, Input, GameEvent
  view/             PixiJS drawing
    scene.ts         backdrop, falling streaks, rocks, player; update(state, alpha)
    rockArt.ts       draws one rock (shape from its id, so it never changes)
    palette.ts       colours
  ui/               DOM screens
    ui.ts            title / HUD / pause / game over / "ad playing" screens
    styles.css, rockfall.css  look
  tests/rockfall.test.ts  rules (movement, collisions, spawning, difficulty)
  tests/lifecycle.test.ts  death, revive, events
  e2e/game.spec.ts     plays the real game in a browser, screenshots
```

(`step.ts` + `types.ts` are the generic "run lifecycle" every small game here starts from; `game.ts` is the part that is specific to RockFall.)

How one run works

1. **Title** screen. `Play` → `startRun()` makes a fresh state: `createState(seed)`.

- Every fixed step (1/60 s) the flow calls `step(state, { targetX })` where `targetX` is the pointer converted to world x. Inside `updateGame` :
 - the player moves toward `targetX` at most `maxSpeed` units/second and is clamped to the field;
 - a spawn timer counts down; when it hits 0 a rock is created (random size, x, sideways drift, fall speed = `fallSpeed(time) ± jitter`) and the timer is reset to `spawnInterval(time)` ;
 - rocks move, bounce off the side walls and are removed once they leave the bottom;
 - collisions: a rock hits if the distance between centres < `(rock.r + player.r) × hitboxScale` (0.85 - forgiving). While `state.involn > 0` (after a revive) nothing hits.
- A hit sets `alive = false` and pushes `{type:'died', score}` onto `state.events` .
- The flow drains events: updates the best score, shows the **game over** screen, offers revive if `!state.reviveUsed && ads.isRewardedReady()` .
- Revive**: `ads.showRewarded()` → if rewarded, `revive(state)` clears the rocks, sets `involn = 1.5` and the game continues. Only once per run (`reviveUsed`).
- Pause**: when the page is hidden (app backgrounded) while playing, the flow pauses; `Resume` continues after `resetClock()` .

Score = `floor(tick / 60)` = whole seconds survived. Time and score derive from the tick count, so they can never drift.

Rendering

`render(alpha)` calls `scene.update(state, alpha)` . For each rock the scene keeps a `Graphics` in a `Map` keyed by `rock.id` : created when a new id appears, positioned at `lerp(prevX, x, alpha)` , destroyed when the id disappears. The player and the scrolling "speed streaks" are the same idea. The engine draws nothing, so the scene paints its own backdrop.

Things to change (with where)

I want to...	Change
Make it easier/harder	<code>sim/config.ts</code> : <code>fall.startSpeed/maxSpeed/gainPerSecond</code> , <code>spawn.startInterval/minInterval/shrinkPerSecond</code> , <code>player.maxSpeed</code> , <code>hitboxScale</code>
Different rock sizes/drift	<code>config.rocks</code>
Another revive rule (e.g. 2 per run)	<code>reviveUsed</code> in <code>sim/step.ts</code> (make it a count) + <code>canRevive</code> in <code>index.ts</code>
New colours / look	<code>view/palette.ts</code> , <code>view/rockArt.ts</code> , <code>ui/rockfall.css</code>
Change score meaning	<code>scoreOf</code> in <code>sim/step.ts</code> , labels in <code>info.ts</code>
Add a new thing that falls (e.g. coin)	add a list to <code>GameData</code> in <code>sim/game.ts</code> + spawn/move/collide in <code>updateGame</code> , a matching event in <code>types.ts</code> , a sprite map in <code>view/scene.ts</code> , a test in <code>tests/rockfall.test.ts</code>

I want to...	Change
Add sound/vibration	in <code>index.ts</code> <code>handleEvent</code> : <code>ctx.haptics.impact('heavy')</code> , <code>ctx.audio.tone({freq:220, endFreq:80, duration:0.3})</code> (see Fruit Slice <code>src/game/sfx.ts</code>)

Exercise: add a "shield" pickup (a good way to learn the layers)

1. **Sim:** in `game.ts` add `pickups: Pickup[]` to `GameData` ; spawn one every ~10 s from `state.rng` ; move it down; if it touches the player set `state.invuln = 3` and push `{type:'shield'}` .
2. **Test:** in `tests/rockfall.test.ts` build a state with a pickup on the player, call `step` , expect `invuln` to be 3 and the event to be drained.
3. **View:** a `Map<id, Graphics>` in `scene.ts` exactly like the rocks.
4. **Flow:** in `handleEvent` play a sound on `'shield'` .
5. `npm run check` , then `npm run e2e` and look at the screenshots.

Run / test / ship

```
cd RockFall
npm run dev          # http://localhost:5173/?seed=42
npm run check        # typecheck + all unit tests
npm run e2e          # browser test + screenshots in e2e/screenshots
npm run android      # build + run on a phone/emulator (JDK 21 + Android SDK)
```

Published as a link by the GitHub Pages workflow (see the engine guide, section 10): github.io/<repo>/rockfall/ . For real ads see section 12 of the engine guide.

Engine updates arrive with `git merge main` (never edit `src/engine/` here).